SAVEETHA SCHOOL OF ENGINEERING
C PROGRAMMING ASSIGNMENT – Logic Analysis and Design Challenges

Field	Entry
Student Name	BAVADARANI S
Register Number	1912260562
Course Code & Name	CSA0209 C PROGRAMMING
Slot	B
Assignment / Task Title	Logic Analysis and Design Challenges (Q1–Q7, 50 marks)
GitHub Repository URL	https://github.com/bavadarini28 /CSA0209-1912260562-BAVADARANI.S.git
Submission Date	12 NOVEMBER 2026

Q7 – Diagnose, Redesign and Validate (chosen program: Q5 prime testing)

A. Diagnose the Bottleneck

Evidence from Q5: for a prime ticket such as 1,000,003 the loop makes 999 divisions, all inside the single loop in is_prime. The count grows with √n (97 → 8, 7919 → 87, 1,000,003 → 999). Verifying every number from 2 to 1,000,003 costs 67,741,504 divisions. Work is repeated unnecessarily: after n % 2 fails, every other even divisor (4, 6, 8 …) cannot divide n either – about half of the divisions are wasted; likewise multiples of 3 (another third). Composite numbers with a small factor (like 100) already exit after one division, so the cost is dominated by primes and by numbers with only large factors.

B. Design Objective

Objective: reduce the number of divisions for large primes (e.g. 1,000,003) by at least 50% and the total for all n ≤ 1,000,003 by at least 45%, with identical results (zero mismatches against the original over 2..1,000,003) and no extra memory.

C. Revised Design

Property exploited: every prime greater than 3 has the form 6k ± 1 (else it is divisible by 2 or 3). Version 1 skips even divisors; Version 2 tests 2 and 3 once, then only candidates 5, 7, 11, 13, 17, 19 … (i and i + 2 with i += 6). Extra memory: none. Trade-off: slightly more code, special cases for n < 4, and one extra test for 2 and 3 per call.

```
is_prime(n):
    if n < 2 return 0 ; if n < 4 return 1
    if n % 2 == 0 return 0 ; if n % 3 == 0 return 0
    i = 5
    while i * i <= n:
        if n % i == 0 or n % (i + 2) == 0: return 0
        i = i + 6
    return 1
int is_prime_6k(unsigned long n)
{
    checks = 0;
    if (n < 2) return 0;
    if (n < 4) return 1;                    /* 2 and 3 */
    checks++; if (n % 2 == 0) return 0;
```

```
  checks++; if (n % 3 == 0) return 0;
  for (unsigned long i = 5; i * i <= n; i += 6) {
    checks++; if (n % i == 0)       return 0;   /* 6k - 1 */
    checks++; if (n % (i + 2) == 0) return 0;   /* 6k + 1 */
  }
  return 1;
}
```

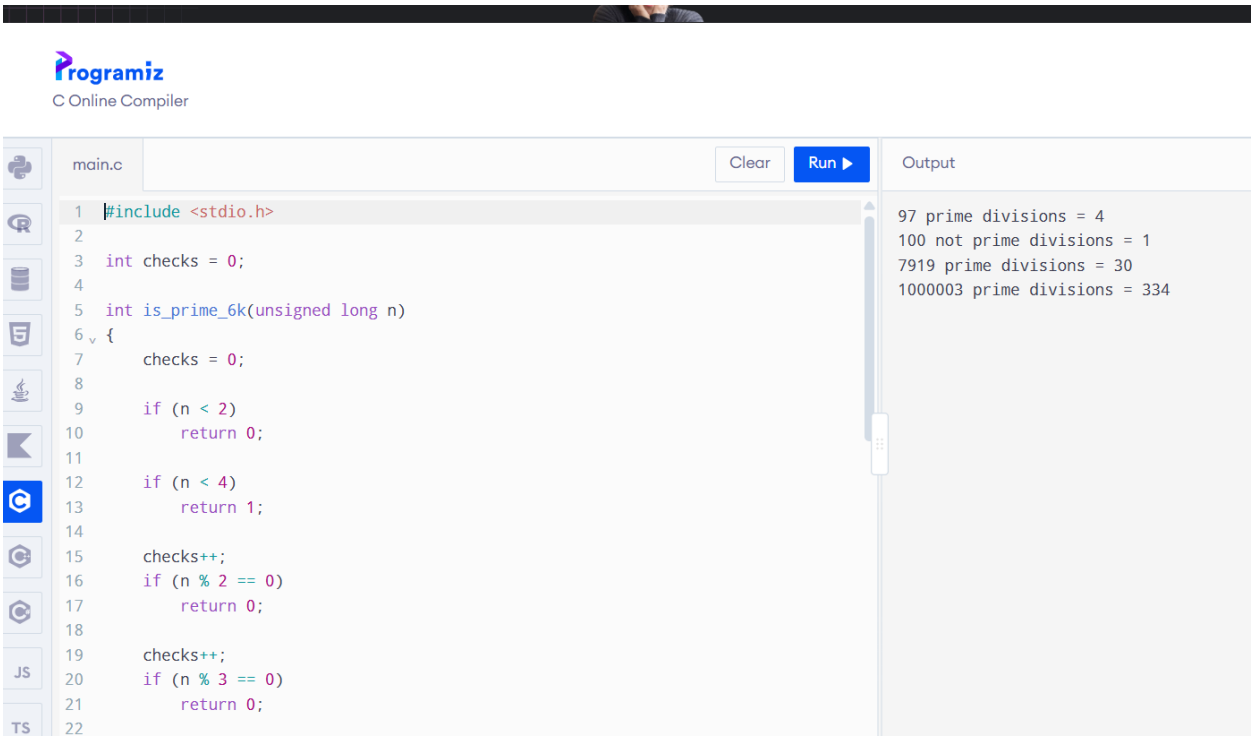
D. Prediction

For 1,000,003: original 999 divisions; odd-only ≈ 999 / 2 ≈ 500 (−50%); 6k ± 1 ≈ 999 / 3 ≈ 333–334 (−67%). Total over all n: roughly −50% and −66%. Measure: number of divisions (and time as a secondary measure). For small or even n (like 100) I expect no gain.

E. Implementation and Validation

n	Prime?	Original	Odd only	6k ± 1
97	yes	8	5	4
100	no	1	1	1
7,919	yes	87	44	30
1,000,003	yes	999	500	334
999,983	yes	998	500	334
1,000,001	no	100	51	35
All n = 2… 1,000,003	78,499 primes	67,741,504	34,601,109	23,739,568
Time for all n (-O0)	–	0.213 s	0.109 s	0.074 s

All three versions agreed on every n from 2 to 1,000,003 (0 mismatches) and found 78,499 primes, which also equals the number found by the sieve. Times are from one run on my machine without optimisation; yours will differ.



F. Defence

- Objective achieved: 999 → 500 (−50.0%) and → 334 (−66.6%) divisions for the large prime; for all n, −48.9% (odd only) and −65.0% (6k ± 1). The odd-only total misses 50% slightly because it spends one extra test on n % 2, and because composites that already exit at i = 2 gain nothing.
- Measured time fell from 0.213 s to 0.074 s (about 2.9× faster), which tracks the drop in divisions (67.7M → 23.7M, about 2.9× fewer): division dominates the run time, so counting divisions is a good proxy for speed.
- New costs: more complex code and special cases (n < 4); more places for an off-by-one error; no extra memory.
- Where the original may still be better: for a few small tickets (n = 100 gains nothing; 97 saves only 4 divisions) its simplicity and obvious correctness outweigh the saving. If many tickets are verified, the sieve (about 2.1 million marking operations and 1 MB for N = 1,000,003) beats all trial-division versions (23.7 million divisions for 6k ± 1), at the price of O(N) memory and a fixed upper limit.
- Conclusion: adopt the 6k ± 1 version for on-demand checks; use the sieve for batch verification of the whole range.

Feedback (CO2)

This assignment helped me apply operators, decision statements and loops to real problems (CO2). I learned to predict results from precedence rules, test boundary values, and judge a design by measured counts, and I reduced the prime-check divisions by about 65% with a 6k±1 redesign.